Actividad 2 — Análisis de reseñas de hoteles en Punta Cana



Puedes ejecutar este notebook localmente (VS Code / Jupyter) o en **Google Colab**.

Si lo abres en Colab, ejecuta primero la celda **Setup para Google Colab** que aparece abajo: instala las dependencias faltantes, descarga el modelo de spaCy, las stopwords de NLTK y baja el dataset `punta_cana.csv` desde el repositorio. En entornos locales esa celda no hace nada y puedes saltarla.

In [4]:
```
# === Setup para Google Colab ===
# Esta celda solo hace algo cuando el notebook se ejecuta en Google Colab.
# En entornos locales (VS Code / Jupyter) se ignora y puede saltarse.
import sys
IN_COLAB = "google.colab" in sys.modules

if IN_COLAB:
    # Dependencias adicionales (Colab ya incluye pandas, sklearn, spacy, nltk)
    # gensim ya no viene preinstalado en Colab; sus binarios requieren numpy 1.x
    !pip install -q unidecode "gensim==4.3.3" "scipy==1.13.1" "numpy<2"

    # Modelo de spaCy en español
    !python -m spacy download es_core_news_md -q

    # Stopwords de NLTK
    import nltk
    nltk.download("stopwords", quiet=True)

    # Descarga del dataset desde el repositorio de GitHub
    import os, urllib.request
    BASE = "https://raw.githubusercontent.com/meendivil/nlp-vision/main/semana-02
    os.makedirs("data", exist_ok=True)
    for archivo in ["punta_cana.csv"]:
        urllib.request.urlretrieve(f"{BASE}/data/{archivo}", f"data/{archivo}")

    print("Setup de Colab completado.")
else:
    print("Entorno local detectado, no se requiere setup adicional.")
```

```
Entorno local detectado, no se requiere setup adicional.
```

In [5]:
```python
# === Carga del dataset ===
import pandas as pd

DATA_PATH = "data/punta_cana.csv"

data = pd.read_csv(DATA_PATH)

print(f"Reseñas: {data.shape[0]:,}  |  Columnas: {data.shape[1]}")
print("Columnas:", list(data.columns))
print("\nValores nulos por columna:")
print(data.isna().sum())

data.head()
```

```
Reseñas: 34,561  |  Columnas: 6
Columnas: ['hotel_name', 'location', 'wrote', 'rating', 'title', 'review_text']

Valores nulos por columna:
hotel_name        0
location       4371
wrote            47
rating            0
title             0
review_text       0
dtype: int64
```

Out[5]:

	hotel_name	location	wrote	rating	title	review_text
0	Secrets Cap Cana Resort & Spa	Villaviciosa, Spain	December 2021	5	Vacaciones en El paraíso	El hotel impecable todo muy limpio las habitac...
1	Secrets Cap Cana Resort & Spa	Villaviciosa, Spain	December 2021	5	HOTEL de la EXCELENCIA	EXCELENTE, magnífico, excepcional, maravilloso...
2	Secrets Cap Cana Resort & Spa	Villaviciosa, Spain	December 2021	5	Sentirse como en CASA!!!!!	Tras conocer bastantes hoteles en distintos lu...
3	Secrets Cap Cana Resort & Spa	Villaviciosa, Spain	December 2021	5	Muy buen nivel	Muy buen hotel. Cumple con todos los requisito...
4	Secrets Cap Cana Resort & Spa	Villaviciosa, Spain	December 2021	5	10	Excelente todo ....sobre todo el personal. Es ...

In [6]:
```python
# === Etiquetado de sentimiento a partir de la calificación ===
# Criterio estándar en la industria hotelera:
#   - Positivas (4 y 5): experiencia satisfactoria o excelente, alta probabilidad
#     de recomendación.
#   - Negativas (1, 2 y 3): 1 y 2 son claramente experiencias negativas; el 3 se
#     agrupa aquí porque una valoración intermedia evidencia áreas de mejora y no
#     una experiencia plenamente satisfactoria.
# En este dataset la calificación está en la columna "rating" y el texto de la
# reseña en "review_text".

COL_RATING = "rating"
COL_TEXTO = "review_text"

# Filtrado del DataFrame por calificación
positivas = data[data[COL_RATING].isin([4, 5])]
negativas = data[data[COL_RATING].isin([1, 2, 3])]

# Extracción del texto de cada grupo como listas de Python
textos_positivos = positivas[COL_TEXTO].dropna().astype(str).tolist()
textos_negativos = negativas[COL_TEXTO].dropna().astype(str).tolist()

total = len(textos_positivos) + len(textos_negativos)
print(f"Positivas (4-5): {len(textos_positivos):,} ({len(textos_positivos) / tota
print(f"Negativas (1-3): {len(textos_negativos):,} ({len(textos_negativos) / tota
print(f"Total etiquetado: {total:,} de {len(data):,} reseñas")

print()
print("Distribución por calificación:")
print(data[COL_RATING].value_counts().sort_index().to_string())

print()
print("Ejemplo positivo:", textos_positivos[0][:160], "...")
print("Ejemplo negativo:", textos_negativos[0][:160], "...")
```

```
Positivas (4-5): 30,632 (88.6%)
Negativas (1-3): 3,929 (11.4%)
Total etiquetado: 34,561 de 34,561 reseñas

Distribución por calificación:
rating
1     1334
2      900
3     1695
4     3924
5    26708

Ejemplo positivo: El hotel impecable todo muy limpio las habitaciones amplias y co
n todo tipo de detalles. Las medidas cocid estaban bien ya que en el bufete tú no
tocas nada tod ...
Ejemplo negativo: Estuvimos una semana en el Hotel. Habitaciones. Amplias y confor
tables. Personal. Atento pero forzado en sus palabras y basado en las propinas esp
ecialmente del ...
```

In [7]:
```python
# === Unificación y organización de datos ===
# La etiqueta de sentimiento se deriva directamente de la calificación, con el
# mismo criterio de la celda anterior: 4-5 -> "positiva", 1-3 -> "negativa".
import numpy as np

data["Sentimiento"] = np.where(data[COL_RATING] >= 4, "positiva", "negativa")

# DataFrame final con las dos columnas que pide la actividad.
# Se conserva "data" intacto (con todas sus columnas) para poder volver a
# consultar rating, hotel u origen durante la interpretación de resultados.
df = data[[COL_TEXTO, "Sentimiento"]].rename(columns={COL_TEXTO: "Comentario"})

print("DataFrame final:", df.shape[0], "filas x", df.shape[1], "columnas")
print("Columnas:", list(df.columns))

print()
print("Reseñas por sentimiento:")
print(df["Sentimiento"].value_counts().to_string())

print()
print("Verificación contra las listas de la celda anterior:")
print(f"  positivas: {(df['Sentimiento'] == 'positiva').sum():,}  |  textos_posit
print(f"  negativas: {(df['Sentimiento'] == 'negativa').sum():,}  |  textos_negat

df.head()
```

```
DataFrame final: 34561 filas x 2 columnas
Columnas: ['Comentario', 'Sentimiento']

Reseñas por sentimiento:
Sentimiento
positiva    30632
negativa     3929

Verificación contra las listas de la celda anterior:
  positivas: 30,632  |  textos_positivos: 30,632
  negativas: 3,929  |  textos_negativos: 3,929
```

Out[7]:

	Comentario	Sentimiento
0	El hotel impecable todo muy limpio las habitac...	positiva
1	EXCELENTE, magnífico, excepcional, maravilloso...	positiva
2	Tras conocer bastantes hoteles en distintos lu...	positiva
3	Muy buen hotel. Cumple con todos los requisito...	positiva
4	Excelente todo ....sobre todo el personal. Es ...	positiva

Paso 1. Preprocesamiento del texto

Sobre el corpus de reseñas se aplican las siguientes operaciones:

- Normalización del texto (conversión a minúsculas).
- Eliminación de palabras vacías (stopwords), conservando las negaciones.
- Lematización.
- Eliminación de caracteres especiales (y números).
- Tokenización.

Este procedimiento reduce el ruido del corpus y prepara los datos para su representación estadística del lenguaje.

In [8]:
```
# === Recursos de lenguaje ===
import spacy
import nltk
from nltk.corpus import stopwords
from unidecode import unidecode

# Descargar lista de stopwords de NLTK (solo la primera vez)
nltk.download("stopwords")

# Cargar modelo lingüístico de spaCy en español
nlp = spacy.load("es_core_news_md")

# Stopwords en español y conservación de negaciones.
# "no", "ni" y "tampoco" se retiran de la lista de stopwords porque invierten la
# polaridad de la reseña ("no recomiendo" vs. "recomiendo"): eliminarlas haría
# indistinguibles dos opiniones opuestas.
stopwords_es = set(stopwords.words("spanish"))
neg_keep = {"no", "ni", "tampoco"}
stopwords_es = stopwords_es - neg_keep

print(f"Stopwords en español: {len(stopwords_es)} (negaciones conservadas: {sorte
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\mendi\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
Stopwords en español: 311 (negaciones conservadas: ['ni', 'no', 'tampoco'])
```

In [10]:

```python
import re


def clean_text(texto: str, return_str: bool = False):
    """
    Limpia un texto en español (Latinoamérica) usando spaCy + NLTK stopwords:
    - Tokenización
    - Elimina espacios y signos de puntuación
    - Elimina números (dígitos y cantidades escritas con letra)
    - Elimina stopwords (usando la lista de NLTK, conservando las negaciones)
    - Lematiza las palabras
    - Convierte a minúsculas y quita acentos
    - Separa los restos de puntuación pegados dentro de un token y descarta los
      fragmentos vacíos o que resulten ser stopwords

    Devuelve una lista de tokens limpios o una cadena si return_str=True.
    """
    doc = nlp(texto)
    tokens = []
    for t in doc:
        if t.is_space or t.is_punct or t.like_num:
            continue
        lema = t.lemma_.lower()
        if lema in stopwords_es:
            continue
        # Cuando falta el espacio tras un signo ("limpio.La"), spaCy entrega un s
        # token: se parte por cualquier carácter que no sea letra o dígito y se v
        # a filtrar cada fragmento (así "limpio.la" queda en "limpio" y se descar
        for fragmento in re.split(r"[\W_]+", lema):
            if not fragmento or fragmento.isdigit() or fragmento in stopwords_es:
                continue
            tokens.append(unidecode(fragmento))
    return " ".join(tokens) if return_str else tokens
```

In [11]:
```
# === Aplicación de la limpieza ===
# Igual que en Tema 2 (data["tokenized_text"] = data["tweet"].apply(clean_text)),
# pero sobre la columna "Comentario" y guardando el resultado en "Tokens".
df["Tokens"] = df["Comentario"].apply(lambda x: clean_text(x))

print("Reseñas tokenizadas:", df["Tokens"].notna().sum())
print("Tokens totales:", int(df["Tokens"].str.len().sum()))
print("Promedio de tokens por reseña:", round(df["Tokens"].str.len().mean(), 1))

df.head()
```

```
Reseñas tokenizadas: 34561
Tokens totales: 1402607
Promedio de tokens por reseña: 40.6
```

Out[11]:

	Comentario	Sentimiento	Tokens
0	El hotel impecable todo muy limpio las habitac...	positiva	[hotel, impecable, limpio, habitacion, amplio,...
1	EXCELENTE, magnífico, excepcional, maravilloso...	positiva	[excelente, magnifico, excepcional, maravillos...
2	Tras conocer bastantes hoteles en distintos lu...	positiva	[tras, conocer, bastante, hotel, distinto, lug...
3	Muy buen hotel. Cumple con todos los requisito...	positiva	[buen, hotel, cumplir, requisito, descansar, s...
4	Excelente todo ....sobre todo el personal. Es ...	positiva	[excelente, personal, ser, diferenciar, resort...

Paso 2. Representación del texto mediante TF-IDF

Para pasar del texto limpio a números se construyen dos representaciones sobre el mismo corpus:

Bag of Words (BoW) — "bolsa de palabras". Cada reseña se convierte en un vector que simplemente cuenta cuántas veces aparece cada término del vocabulario; el orden de las palabras se descarta (de ahí lo de "bolsa"). El resultado es una **matriz documento-término**: una fila por reseña, una columna por término. Es la base de la que parte TF-IDF, y su límite es que una palabra que sale en todas las reseñas (`hotel`) pesa tanto como una que distingue de verdad a una reseña.

TF-IDF con bigramas — toma esos mismos conteos (*term frequency*) y los pondera a la baja según en cuántos documentos aparece el término (*inverse document frequency*). Así, lo que sale en todas partes pierde peso y lo característico de cada reseña lo gana. Se usa `ngram_range=(1, 2)` , de modo que el vocabulario incluye tanto palabras sueltas (`limpio`) como secuencias de dos palabras (`no limpio` , `buena ubicacion`), que es lo que permite captar expresiones que una palabra aislada no representa. `max_features=5000` acota el vocabulario a los 5,000 términos más frecuentes, porque al añadir bigramas crece muy rápido.

La celda de BoW se incluye como contraste: sus conteos en crudo muestran qué cambia al ponderar por idf.

In [12]:
```
# === Bag of Words: matriz documento-término ===
from itertools import chain
from sklearn.feature_extraction.text import CountVectorizer

# La columna "Tokens" ya viene tokenizada, sin stopwords y lematizada, así que se
# le indica al vectorizador que NO vuelva a procesar el texto: el analizador de
# identidad recibe la lista de tokens y la devuelve tal cual.
# min_df=5 descarta los términos que aparecen en menos de 5 reseñas (cola de ruid
# erratas, nombres propios sueltos).
identidad = lambda tokens: tokens

bow_vectorizer = CountVectorizer(analyzer=identidad, min_df=5)
X_bow = bow_vectorizer.fit_transform(df["Tokens"])
vocabulario = bow_vectorizer.get_feature_names_out()

vocab_sin_filtro = len(set(chain.from_iterable(df["Tokens"])))
densidad = X_bow.nnz / (X_bow.shape[0] * X_bow.shape[1])

print(f"Matriz documento-término: {X_bow.shape[0]:,} documentos x {X_bow.shape[1]
print(f"Vocabulario: {len(vocabulario):,} términos (de {vocab_sin_filtro:,} sin e
print(f"Celdas distintas de cero: {X_bow.nnz:,} ({densidad:.3%} de la matriz)")
print("La matriz se mantiene dispersa: en denso ocuparía",
      f"{X_bow.shape[0] * X_bow.shape[1] * 8 / 1e9:.1f} GB.")

# Términos más frecuentes en todo el corpus según la matriz
conteos = np.asarray(X_bow.sum(axis=0)).ravel()
orden_bow = conteos.argsort()[::-1]

print()
print("Top 10 por conteo (Bag of Words):")
for i in orden_bow[:10]:
    print(f"  {vocabulario[i]:<15} {conteos[i]:>8,}")

# Vista de un recorte de la matriz (solo se densifica lo que se va a mostrar)
top8 = orden_bow[:8]
vista = pd.DataFrame(X_bow[:5].toarray()[:, top8], columns=vocabulario[top8])
vista.insert(0, "Sentimiento", df["Sentimiento"].head(5).values)
vista
```

```
Matriz documento-término: 34,561 documentos x 9,766 términos
Vocabulario: 9,766 términos (de 40,235 sin el filtro min_df)
Celdas distintas de cero: 1,164,895 (0.345% de la matriz)
La matriz se mantiene dispersa: en denso ocuparía 2.7 GB.

Top 10 por conteo (Bag of Words):
  ser              68,251
  hotel            31,617
  no               22,329
  excelente        19,620
  buen             17,704
  servicio         17,592
  haber            16,999
  comida           15,756
  habitacion       15,202
  tener            15,198
```

Out[12]:

	Sentimiento	ser	hotel	no	excelente	buen	servicio	haber	comida
0	positiva	2	1	2	0	0	0	1	0
1	positiva	1	3	1	1	0	0	3	0
2	positiva	6	2	0	0	0	3	0	0
3	positiva	3	1	0	1	1	0	1	0
4	positiva	4	0	0	1	0	0	0	0

In [13]:

```python
# === TF-IDF con bigramas: ponderación por rareza ===
from sklearn.feature_extraction.text import TfidfVectorizer

# El paso pide bigramas, y "ngram_range" NO se aplica cuando "analyzer" es un
# callable: scikit-learn lo ignora y avisa ("the parameter 'ngram_range' will not
# be used since 'analyzer' is callable"). Por eso los tokens ya limpios se vuelve
# a unir en una cadena y se deja que el analizador por defecto forme los n-gramas
df["Texto_limpio"] = df["Tokens"].str.join(" ")

# ngram_range=(1, 2) -> unigramas ("limpio") y bigramas ("no limpio").
# max_features=5000  -> con bigramas el vocabulario se dispara, así que se conser
#                       los 5,000 términos más frecuentes del corpus.
# Nota: el token_pattern por defecto descarta los tokens de una sola letra.
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X_tfidf = tfidf_vectorizer.fit_transform(df["Texto_limpio"])
terminos = tfidf_vectorizer.get_feature_names_out()

n_bigramas = sum(1 for t in terminos if " " in t)
print(f"Matriz TF-IDF: {X_tfidf.shape[0]:,} x {X_tfidf.shape[1]:,}")
print(f"  unigramas: {len(terminos) - n_bigramas:,}  |  bigramas: {n_bigramas:,}"

# Términos con mayor peso DENTRO de un documento concreto
def terminos_con_mayor_peso(posicion, n=8):
    fila = X_tfidf[posicion]
    pares = sorted(zip(fila.indices, fila.data), key=lambda par: -par[1])[:n]
    return [(terminos[i], round(float(peso), 3)) for i, peso in pares]

for etiqueta in ("positiva", "negativa"):
    posicion = int(np.where(df["Sentimiento"].values == etiqueta)[0][0])
    print()
    print(f"--- Reseña {etiqueta} (fila {posicion}) ---")
    print(df["Comentario"].iloc[posicion][:200], "...")
    print("Términos con mayor peso:")
    for termino, peso in terminos_con_mayor_peso(posicion):
        print(f"  {termino:<22} {peso:.3f}")

# Bigramas que más pesan en promedio: las secuencias léxicas que caracterizan al
peso_medio = np.asarray(X_tfidf.mean(axis=0)).ravel()
orden_tfidf = peso_medio.argsort()[::-1]
solo_bigramas = [j for j in orden_tfidf if " " in terminos[j]][:15]

print()
print("Top 15 BIGRAMAS por peso medio TF-IDF:")
for j in solo_bigramas:
    print(f"  {terminos[j]:<28} {peso_medio[j]:.4f}")

# Contraste global: qué sube y qué baja al ponderar por idf.
# El BoW de la celda anterior es solo de unigramas, así que la columna TF-IDF pue
# incluir bigramas que por definición no existen en el conteo de la izquierda.
print()
print(f"{'Top 10 por CONTEO (BoW)':<32}{'Top 10 por PESO MEDIO (TF-IDF)'}")
for i, j in zip(orden_bow[:10], orden_tfidf[:10]):
    print(f"  {vocabulario[i]:<14}{conteos[i]:>8,}        {terminos[j]:<20}{peso_
```

```
Matriz TF-IDF: 34,561 x 5,000
  unigramas: 2,400  |  bigramas: 2,600

--- Reseña positiva (fila 0) ---
El hotel impecable todo muy limpio las habitaciones amplias y con todo tipo de det
alles. Las medidas cocid estaban bien ya que en el bufete tú no tocas nada todo te
lo sirven ellos . En la piscina y e ...
Términos con mayor peso:
  bufete                0.290
  servir                0.245
  hotel impecable       0.184
  continuamente         0.184
  paella                0.181
  amable divertido      0.177
  playa                 0.175
  mr blue               0.174

--- Reseña negativa (fila 7) ---
Estuvimos una semana en el Hotel. Habitaciones. Amplias y confortables. Personal.
Atento pero forzado en sus palabras y basado en las propinas especialmente del tur
ista americano Comida. Mucha de ella ...
Términos con mayor peso:
  americano             0.413
  tipo comida           0.309
  turismo               0.287
  tipo                  0.195
  referencia            0.156
  caracteristica        0.154
  europeo               0.152
  decepcionar           0.149

Top 15 BIGRAMAS por peso medio TF-IDF:
  ser excelente               0.0117
  hotel ser                   0.0115
  ser mejor                   0.0095
  excelente servicio          0.0091
  ser buen                    0.0084
  punta cana                  0.0084
  buen servicio               0.0077
  no ser                      0.0072
  comida ser                  0.0071
  excelente atencion          0.0066
  no haber                    0.0063
  haber ser                   0.0063
  habitacion ser              0.0062
  personal ser                0.0060
  atencion personal           0.0058

Top 10 por CONTEO (BoW)         Top 10 por PESO MEDIO (TF-IDF)
  ser             68,251        ser                   0.0707
  hotel           31,617        hotel                 0.0403
  no              22,329        excelente             0.0354
  excelente       19,620        no                    0.0334
  buen            17,704        buen                  0.0330
  servicio        17,592        servicio              0.0312
  haber           16,999        haber                 0.0285
  comida          15,756        atencion              0.0273
  habitacion      15,202        comida                0.0266
  tener           15,198        habitacion            0.0254
```

Paso 3. Entrenamiento del modelo: regresión logística

La matriz TF-IDF del paso anterior es la entrada (`X`) y la etiqueta de sentimiento es la salida (`y`). El corpus se parte en 80% entrenamiento / 20% prueba y se ajusta una regresión logística, que aprende un peso por término: positivo si empuja la reseña hacia *positiva*, negativo si la empuja hacia *negativa*.

Dos observaciones sobre este corpus:

- Está **desbalanceado**: 88.6% de las reseñas son positivas. Por eso el split usa `stratify=y` , para que entrenamiento y prueba conserven esa misma proporción.
- Por el mismo desbalance, la **exactitud engaña**: un clasificador que respondiera siempre "positiva" ya acertaría ~88.6%. La celda imprime esa línea base junto a la del modelo para poder juzgarla.

In [14]:

```python
# === Entrenamiento: regresión logística sobre TF-IDF ===
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X = X_tfidf            # matriz TF-IDF (unigramas + bigramas)
y = df["Sentimiento"]  # etiqueta: "positiva" / "negativa"

# stratify=y conserva la proporción real de clases en ambas particiones; sin él,
# el muestreo aleatorio podría dejar el conjunto de prueba con una mezcla distint
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

print(f"Entrenamiento: {X_train.shape[0]:,} reseñas  |  Prueba: {X_test.shape[0]:
print("Proporción de clases en entrenamiento:")
print(y_train.value_counts(normalize=True).round(3).to_string())

# max_iter=1000 porque el solver por defecto (lbfgs) no converge en 100 iteracion
# con un espacio de 5,000 dimensiones.
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# La evaluación por clase va en el paso siguiente; aquí solo se comprueba que el
# modelo aprendió algo por encima de responder siempre la clase mayoritaria.
base = (y_test == "positiva").mean()
print()
print(f"Exactitud en entrenamiento : {model.score(X_train, y_train):.3f}")
print(f"Exactitud en prueba        : {model.score(X_test, y_test):.3f}")
print(f"Línea base (todo positiva) : {base:.3f}")

# Qué aprendió: los términos con mayor peso hacia cada clase.
# model.classes_ viene ordenado alfabéticamente -> coeficiente positivo = "positi
coeficientes = model.coef_[0]
orden_coef = coeficientes.argsort()

print()
print(f"(coeficiente > 0 empuja a '{model.classes_[1]}', < 0 a '{model.classes_[0
print()
print("Términos que más empujan a NEGATIVA:")
for j in orden_coef[:10]:
    print(f"  {terminos[j]:<28} {coeficientes[j]:>7.3f}")
print()
print("Términos que más empujan a POSITIVA:")
for j in orden_coef[::-1][:10]:
    print(f"  {terminos[j]:<28} {coeficientes[j]:>7.3f}")
```

```
Entrenamiento: 27,648 reseñas  |  Prueba: 6,913 reseñas
Proporción de clases en entrenamiento:
Sentimiento
positiva    0.886
negativa    0.114

Exactitud en entrenamiento : 0.962
Exactitud en prueba        : 0.952
Línea base (todo positiva) : 0.886

(coeficiente > 0 empuja a 'positiva', < 0 a 'negativa')

Términos que más empujan a NEGATIVA:
  no                          -6.826
  malo                        -5.463
  pesimo                      -5.463
  peor                        -4.784
  mal                         -4.512
  mala                        -3.901
  ni                          -3.498
  horrible                    -3.260
  regular                     -3.210
  no volver                   -3.153

Términos que más empujan a POSITIVA:
  excelente                    6.990
  gracias                      4.003
  perfecto                     3.083
  maravilloso                  3.076
  especial                     3.018
  encantar                     2.961
  mejor                        2.898
  genial                       2.758
  recomendar                   2.724
  buen                         2.717
```

Paso 4. Evaluación del modelo

Se evalúa sobre el 20% de prueba, que el modelo no vio durante el entrenamiento:

- **Accuracy** — porcentaje total de aciertos. Con un corpus 88.6% / 11.4% es la métrica menos informativa: responder siempre "positiva" ya la deja en ~0.886.
- **Precision** (por clase) — de lo que el modelo marcó como esa clase, cuánto acertó.
- **Recall** (por clase) — de lo que realmente era esa clase, cuánto encontró. En este corpus es la métrica clave: mide si el modelo detecta las reseñas negativas o las pasa por alto.
- **F1-score** — media armónica de precision y recall; resume ambas en un número.
- **Matriz de confusión** — el desglose de aciertos y errores por celda, que es donde el desbalance se vuelve visible.

In [15]:
```python
# === Evaluación del modelo base ===
from sklearn.metrics import accuracy_score, classification_report, confusion_matr

y_pred = model.predict(X_test)

print("=== Modelo base: LogisticRegression(max_iter=1000) ===")
print()
print(classification_report(y_test, y_pred, digits=3))

etiquetas = list(model.classes_)  # ['negativa', 'positiva']
cm = confusion_matrix(y_test, y_pred, labels=etiquetas)
cm_df = pd.DataFrame(
    cm,
    index=[f"real: {e}" for e in etiquetas],
    columns=[f"pred: {e}" for e in etiquetas],
)
print("Matriz de confusión:")
print(cm_df.to_string())

# Lectura dirigida: qué pasa con la clase minoritaria
negativas_reales = cm[0].sum()
negativas_detectadas = cm[0, 0]
base = (y_test == "positiva").mean()

print()
print(f"Accuracy del modelo        : {accuracy_score(y_test, y_pred):.3f}")
print(f"Línea base (todo positiva) : {base:.3f}")
print(
    f"Reseñas negativas detectadas: {negativas_detectadas:,} de {negativas_reales
    f"({negativas_detectadas / negativas_reales:.1%})"
)
```

```
=== Modelo base: LogisticRegression(max_iter=1000) ===

              precision    recall  f1-score   support

    negativa      0.873     0.673     0.760       786
    positiva      0.959     0.987     0.973      6127

    accuracy                          0.952      6913
   macro avg      0.916     0.830     0.867      6913
weighted avg      0.949     0.952     0.949      6913

Matriz de confusión:
                pred: negativa  pred: positiva
real: negativa             529             257
real: positiva              77            6050

Accuracy del modelo        : 0.952
Línea base (todo positiva) : 0.886
Reseñas negativas detectadas: 529 de 786 (67.3%)
```

Corrección del desbalance

El modelo base optimiza el error total, y con 88.6% de reseñas positivas la forma más barata de equivocarse poco es inclinarse hacia "positiva". El resultado es una accuracy alta con un recall pobre en la clase negativa: justo el problema que el paso pide identificar.

`class_weight="balanced"` corrige eso pesando cada clase por el inverso de su frecuencia: como las negativas son ~8 veces menos numerosas, cada error sobre una negativa pasa a costar ~8 veces más. El modelo deja de poder ignorarlas.

El intercambio esperado: **sube el recall de negativas y baja su precision** (al marcar más reseñas como negativas, se cuelan falsos positivos), y la accuracy total suele bajar unos puntos. Por eso la comparación se hace sobre F1 y recall de la clase minoritaria, no sobre accuracy.

In [16]:

```python
# === Corrección del desbalance: class_weight="balanced" ===
from sklearn.metrics import f1_score, precision_recall_fscore_support

# Mismo split y mismos hiperparámetros; lo único que cambia es el peso por clase.
model_bal = LogisticRegression(max_iter=1000, class_weight="balanced")
model_bal.fit(X_train, y_train)
y_pred_bal = model_bal.predict(X_test)

print("=== Modelo balanceado: class_weight='balanced' ===")
print()
print(classification_report(y_test, y_pred_bal, digits=3))

cm_bal = confusion_matrix(y_test, y_pred_bal, labels=etiquetas)
cm_bal_df = pd.DataFrame(
    cm_bal,
    index=[f"real: {e}" for e in etiquetas],
    columns=[f"pred: {e}" for e in etiquetas],
)
print("Matriz de confusión:")
print(cm_bal_df.to_string())

# --- Comparación lado a lado ---
def resumen(nombre, y_true, y_hat):
    p, r, f, _ = precision_recall_fscore_support(
        y_true, y_hat, labels=["negativa"], zero_division=0
    )
    return {
        "Modelo": nombre,
        "Accuracy": accuracy_score(y_true, y_hat),
        "Precision (neg)": p[0],
        "Recall (neg)": r[0],
        "F1 (neg)": f[0],
        "F1 macro": f1_score(y_true, y_hat, average="macro"),
    }

comparacion = pd.DataFrame([
    resumen("Base", y_test, y_pred),
    resumen("Balanced", y_test, y_pred_bal),
]).set_index("Modelo")

print()
print("Comparación (clase 'negativa' = la minoritaria):")
print(comparacion.round(3).to_string())

detectadas_bal = cm_bal[0, 0]
print()
print(
    f"Negativas detectadas: {negativas_detectadas:,} -> {detectadas_bal:,} "
    f"de {negativas_reales:,}"
)
```

```
=== Modelo balanceado: class_weight='balanced' ===

              precision    recall  f1-score   support

    negativa      0.661     0.910     0.766       786
    positiva      0.988     0.940     0.963      6127

    accuracy                          0.937      6913
   macro avg      0.825     0.925     0.865      6913
weighted avg      0.951     0.937     0.941      6913

Matriz de confusión:
                pred: negativa  pred: positiva
real: negativa             715              71
real: positiva             366            5761

Comparación (clase 'negativa' = la minoritaria):
          Accuracy  Precision (neg)  Recall (neg)  F1 (neg)  F1 macro
Modelo
Base         0.952            0.873         0.673     0.760     0.867
Balanced     0.937            0.661         0.910     0.766     0.865

Negativas detectadas: 529 -> 715 de 786
```

In [17]:

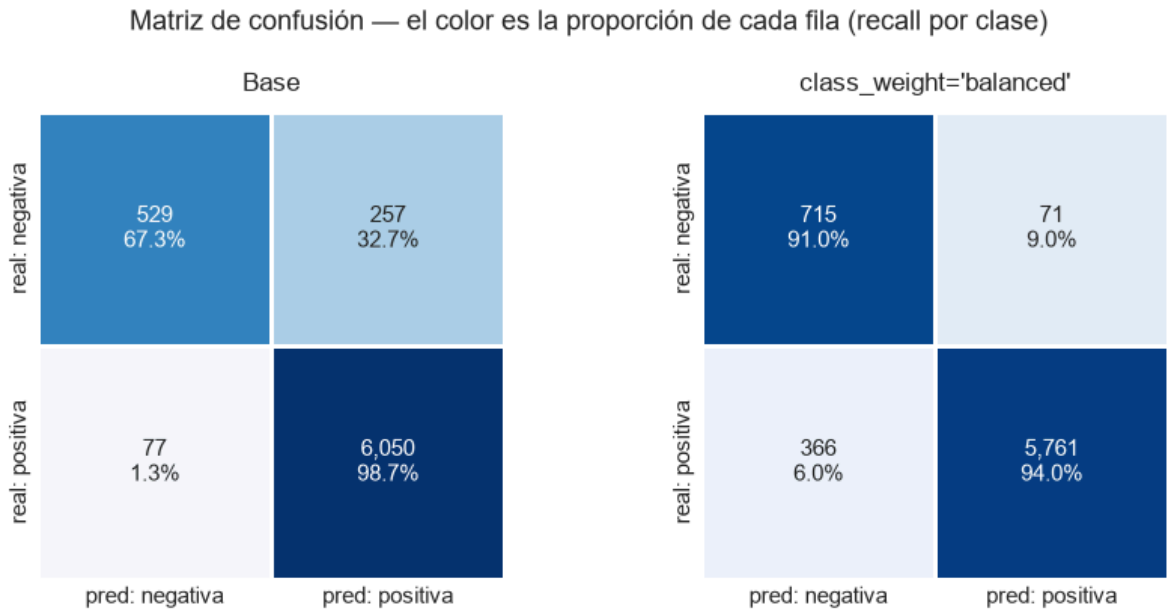
```python
# === Matriz de confusión: base vs. balanceado ===
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="white")

# El color codifica la PROPORCIÓN POR FILA (es decir, el recall de cada clase), n
# conteo: con 7,867 positivas frente a 786 negativas, un mapa de conteos pintaría
# sola celda oscura y ocultaría justo lo que se quiere ver. La escala 0-1 es la m
# en ambos paneles, así que se pueden comparar directamente.
fig, axes = plt.subplots(1, 2, figsize=(9, 4.2))

for ax, (titulo, matriz) in zip(axes, [
    ("Base", cm),
    ("class_weight='balanced'", cm_bal),
]):
    proporcion = matriz / matriz.sum(axis=1, keepdims=True)
    anotaciones = [
        [f"{matriz[i, j]:,}\n{proporcion[i, j]:.1%}" for j in range(matriz.shape[
        for i in range(matriz.shape[0])
    ]
    sns.heatmap(
        proporcion,
        annot=anotaciones,
        fmt="",
        cmap="Blues",          # secuencial de un solo tono: claro = poco, oscuro
        vmin=0, vmax=1,
        cbar=False,
        square=True,
        linewidths=2,
        linecolor="white",
        xticklabels=[f"pred: {e}" for e in etiquetas],
        yticklabels=[f"real: {e}" for e in etiquetas],
        annot_kws={"fontsize": 11},
        ax=ax,
    )
    ax.set_title(titulo, fontsize=12, pad=10)
    ax.tick_params(length=0)

fig.suptitle(
    "Matriz de confusión — el color es la proporción de cada fila (recall por cla
    fontsize=13,
)
fig.tight_layout()
plt.show()
```



Paso 5. Implementación de un modelo basado en Transformers

Hasta aquí el clasificador se construyó desde cero: se limpió el texto, se contaron términos y se ajustó una regresión logística sobre esas cuentas. Un modelo basado en Transformers invierte el planteamiento — ya viene entrenado, y se usa tal cual.

Qué modelo y por qué. El ejemplo de la actividad muestra `bert-base-uncased`, que sirve para ilustrar la API pero no para esta tarea: está entrenado en inglés y no tiene cabeza de clasificación de sentimiento (`pipeline` le añadiría una con pesos aleatorios, así que sus predicciones serían ruido). Se usa en su lugar **`finiteautomata/beto-sentiment-analysis`**: BETO es BERT entrenado en español, y esta variante ya está afinada para análisis de sentimiento.

Tres diferencias frente al modelo clásico:

- **No hay entrenamiento.** No hay `fit`, ni split, ni vocabulario que construir. Se carga y se predice.
- **Recibe el texto crudo.** Su propio tokenizador (WordPiece) se encarga del texto; no se le pasa la salida del Paso 1. Lematizar y quitar stopwords antes le quitaría justamente el contexto que sabe aprovechar.
- **Lee el orden de las palabras.** "no recomiendo" y "recomiendo" son vectores idénticos en bag of words; para un Transformer son frases distintas.

In [18]:
```python
# === Carga del modelo preentrenado ===
from transformers import pipeline
import torch

MODELO = "finiteautomata/beto-sentiment-analysis"

# device=0 usa la primera GPU; -1 fuerza CPU. La descarga (~440 MB) ocurre solo l
# primera vez y queda en la caché local de Hugging Face.
usa_gpu = torch.cuda.is_available()
device = 0 if usa_gpu else -1
print("Dispositivo:", torch.cuda.get_device_name(0) if usa_gpu else "CPU")

classifier = pipeline("sentiment-analysis", model=MODELO, device=device)

print("Etiquetas del modelo:", classifier.model.config.id2label)

# Prueba rápida antes de lanzarlo sobre el corpus
ejemplos = [
    "El hotel impecable, todo muy limpio y el personal amabilisimo.",
    "Pesimo servicio, la habitacion estaba sucia y la comida fria.",
    "El hotel esta bien, nada del otro mundo.",
    "No recomiendo este hotel.",
]
for texto, r in zip(ejemplos, classifier(ejemplos)):
    print(f"  {r['label']:4s} {r['score']:.3f}  <- {texto}")
```

```
Dispositivo: NVIDIA GeForce RTX 5070 Ti Laptop GPU
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_T
OKEN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Etiquetas del modelo: {0: 'NEG', 1: 'NEU', 2: 'POS'}
  POS  0.997  <- El hotel impecable, todo muy limpio y el personal amabilisimo.
  NEG  0.999  <- Pesimo servicio, la habitacion estaba sucia y la comida fria.
  POS  0.998  <- El hotel esta bien, nada del otro mundo.
  NEG  0.980  <- No recomiendo este hotel.
```

In [24]:
```python
# === Inferencia sobre el conjunto de prueba ===
import time

# Se usan exactamente las mismas reseñas que evaluaron a la regresión logística:
# y_test conserva los índices originales de df, así que basta recuperar su texto.
# Nota: se le pasa el COMENTARIO CRUDO, no los tokens del Paso 1.
textos_test = df.loc[y_test.index, "Comentario"].astype(str).tolist()
print(f"Reseñas a clasificar: {len(textos_test):,}")

# truncation + max_length: BETO admite 512 posiciones; 256 cubre la gran mayoría
# las reseñas y reduce el cómputo a la mitad.
# Cada modelo tiene su propio limite de longitud y hay que respetarlo: BETO admit
# 512 posiciones, pero RoBERTuito (entrenado con tuits) solo 128. Pasar un max_le
# mayor al del modelo produce indices de posicion fuera de rango, que en GPU abor
# con "scatter gather kernel index out of bounds".
def longitud_maxima(clf, tope=256):
    return min(tope, clf.tokenizer.model_max_length)


t0 = time.time()
predicciones = classifier(
    textos_test,
    batch_size=64 if usa_gpu else 16,
    truncation=True,
    max_length=longitud_maxima(classifier),
)
transcurrido = time.time() - t0
print(f"Tiempo: {transcurrido:.1f}s  ({len(textos_test)/transcurrido:,.0f} reseña

# --- De 3 clases a 2 ---
# BETO devuelve POS / NEU / NEG, pero la etiqueta de este corpus es binaria. NEU
# mapea a "negativa" por coherencia con el criterio del Paso 1, donde la califica
# de 3 estrellas (la intermedia) también se contó como negativa.
MAPEO = {"POS": "positiva", "NEU": "negativa", "NEG": "negativa"}
y_pred_beto = pd.Series([MAPEO[p["label"]] for p in predicciones], index=y_test.i

conteo_crudo = pd.Series([p["label"] for p in predicciones]).value_counts()
print()
print("Distribución de las 3 etiquetas originales del modelo:")
print(conteo_crudo.to_string())
```

```
Reseñas a clasificar: 6,913
Tiempo: 28.5s  (242 reseñas/s)

Distribución de las 3 etiquetas originales del modelo:
POS    5854
NEG     623
NEU     436
```

In [25]:
```
# === Evaluación de BETO y comparación con el modelo clásico ===
print("=== BETO (preentrenado, sin ajuste sobre este corpus) ===")
print()
print(classification_report(y_test, y_pred_beto, digits=3))

cm_beto = confusion_matrix(y_test, y_pred_beto, labels=etiquetas)
print("Matriz de confusión:")
print(pd.DataFrame(
    cm_beto,
    index=[f"real: {e}" for e in etiquetas],
    columns=[f"pred: {e}" for e in etiquetas],
).to_string())

# resumen() viene de la celda de corrección del desbalance
comparacion_3 = pd.DataFrame([
    resumen("LogReg base", y_test, y_pred),
    resumen("LogReg balanced", y_test, y_pred_bal),
    resumen("BETO", y_test, y_pred_beto),
]).set_index("Modelo")

print()
print("Comparación de los tres modelos (clase 'negativa' = la minoritaria):")
print(comparacion_3.round(3).to_string())
```

```
=== BETO (preentrenado, sin ajuste sobre este corpus) ===

              precision    recall  f1-score   support

    negativa      0.620     0.836     0.712       786
    positiva      0.978     0.934     0.956      6127

    accuracy                          0.923      6913
   macro avg      0.799     0.885     0.834      6913
weighted avg      0.937     0.923     0.928      6913

Matriz de confusión:
                pred: negativa  pred: positiva
real: negativa             657             129
real: positiva             402            5725

Comparación de los tres modelos (clase 'negativa' = la minoritaria):
                 Accuracy  Precision (neg)  Recall (neg)  F1 (neg)  F1 macro
Modelo
LogReg base         0.952            0.873         0.673     0.760     0.867
LogReg balanced     0.937            0.661         0.910     0.766     0.865
BETO                0.923            0.620         0.836     0.712     0.834
```

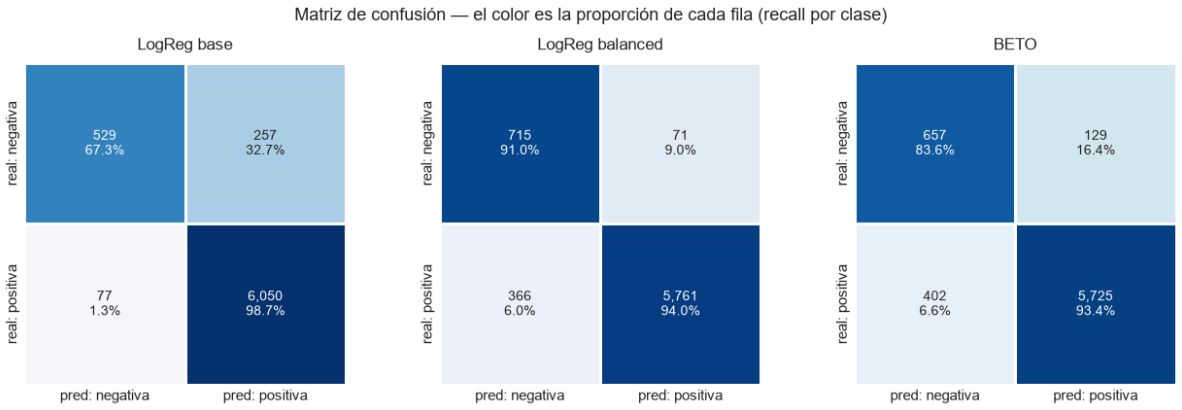
In [26]:

```
# === Matriz de confusión de los tres modelos ===
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))

for ax, (titulo, matriz) in zip(axes, [
    ("LogReg base", cm),
    ("LogReg balanced", cm_bal),
    ("BETO", cm_beto),
]):
    proporcion = matriz / matriz.sum(axis=1, keepdims=True)
    anotaciones = [
        [f"{matriz[i, j]:,}\n{proporcion[i, j]:.1%}" for j in range(matriz.shape[
        for i in range(matriz.shape[0])
    ]
    sns.heatmap(
        proporcion,
        annot=anotaciones,
        fmt="",
        cmap="Blues",
        vmin=0, vmax=1,
        cbar=False,
        square=True,
        linewidths=2,
        linecolor="white",
        xticklabels=[f"pred: {e}" for e in etiquetas],
        yticklabels=[f"real: {e}" for e in etiquetas],
        annot_kws={"fontsize": 10},
        ax=ax,
    )
    ax.set_title(titulo, fontsize=12, pad=10)
    ax.tick_params(length=0)

fig.suptitle(
    "Matriz de confusión — el color es la proporción de cada fila (recall por cla
    fontsize=13,
)
fig.tight_layout()
plt.show()
```



Generación de predicciones

Con el modelo ya cargado, clasificar un texto es una sola llamada. El `pipeline` devuelve, por cada texto, un diccionario con dos campos:

- **`label`** — la etiqueta de sentimiento predicha.
- **`score`** — la probabilidad asociada, entre 0 y 1, que indica qué tan seguro está el modelo.

Se usa un conjunto fijo de textos de prueba que incluye casos fáciles, una negación, una reseña ambigua, una reseña real del corpus y una frase en inglés. Ese mismo conjunto se le pasará después a los otros dos modelos, para que la comparación sea sobre el mismo material.

In [27]:
```
# === Generación de predicciones con BETO ===
# Forma mínima: un texto, una llamada.
text = "El servicio fue excelente y el personal muy amable."
result = classifier(text)
print(result)

# Conjunto de prueba que se reutilizará con los tres modelos.
TEXTOS_DEMO = [
    "El servicio fue excelente y el personal muy amable.",
    "Pesimo servicio, la habitacion estaba sucia y la comida fria.",
    "No recomiendo este hotel.",
    "El hotel esta bien, nada del otro mundo.",
    df["Comentario"].iloc[0][:200],  # reseña real del corpus
    "The service was excellent and the staff was very friendly.",
]


def mostrar_predicciones(clf, textos=TEXTOS_DEMO, nombre=""):
    """Imprime etiqueta y probabilidad de cada texto, y devuelve la lista de resu
    resultados = clf(textos, truncation=True, max_length=longitud_maxima(clf))
    if nombre:
        print(f"--- {nombre} ---")
    for texto, r in zip(textos, resultados):
        print(f"  {r['label']:9s} {r['score']:.3f}  <- {texto[:70]}")
    return resultados


print()
pred_beto = mostrar_predicciones(classifier, nombre=MODELO)
```

```
[{'label': 'POS', 'score': 0.9984838366508484}]

--- finiteautomata/beto-sentiment-analysis ---
  POS       0.998  <- El servicio fue excelente y el personal muy amable.
  NEG       0.999  <- Pesimo servicio, la habitacion estaba sucia y la comida fri
a.
  NEG       0.980  <- No recomiendo este hotel.
  POS       0.998  <- El hotel esta bien, nada del otro mundo.
  POS       0.993  <- El hotel impecable todo muy limpio las habitaciones amplias
y con todo
  POS       0.993  <- The service was excellent and the staff was very friendly.
```

Efecto del modelo elegido: dos en español y uno en inglés

Se repite exactamente el mismo ejercicio con otros dos modelos preentrenados:

Modelo	Idioma de entrenamiento	Etiquetas
`finiteautomata/beto-sentiment-analysis`	español	NEG / NEU / POS
`pysentimiento/robertuito-sentiment-analysis`	español (texto de redes sociales)	NEG / NEU / POS
`distilbert-base-uncased-finetuned-sst-2-english`	**inglés** (reseñas de cine, SST-2)	NEGATIVE / POSITIVE

Los dos primeros deberían comportarse de forma parecida. El tercero es el experimento interesante: está afinado para análisis de sentimiento, pero **en un idioma distinto al del corpus**. Su tokenizador parte el español en fragmentos que nunca vio durante el entrenamiento, así que sus predicciones sobre estas reseñas no tienen por qué significar nada — aunque el `score` siga saliendo alto, porque un modelo siempre reparte probabilidad entre las clases que conoce.

Esa es la lección del paso: **una probabilidad alta no es evidencia de que el modelo entienda el texto.**

In [28]:
```
# === Modelo 2: RoBERTuito (español) ===
MODELO_ROBERTUITO = "pysentimiento/robertuito-sentiment-analysis"

clf_robertuito = pipeline("sentiment-analysis", model=MODELO_ROBERTUITO, device=d
print("Etiquetas del modelo:", clf_robertuito.model.config.id2label)
print()
pred_robertuito = mostrar_predicciones(clf_robertuito, nombre=MODELO_ROBERTUITO)
```

```
Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Etiquetas del modelo: {0: 'NEG', 1: 'NEU', 2: 'POS'}

--- pysentimiento/robertuito-sentiment-analysis ---
  POS       0.977  <- El servicio fue excelente y el personal muy amable.
  NEG       0.967  <- Pesimo servicio, la habitacion estaba sucia y la comida fri
a.
  NEG       0.791  <- No recomiendo este hotel.
  POS       0.867  <- El hotel esta bien, nada del otro mundo.
  POS       0.945  <- El hotel impecable todo muy limpio las habitaciones amplias
y con todo
  POS       0.941  <- The service was excellent and the staff was very friendly.
```

In [29]:
```python
# === Modelo 3: DistilBERT afinado en inglés (SST-2) ===
MODELO_DISTIL = "distilbert-base-uncased-finetuned-sst-2-english"

clf_distil = pipeline("sentiment-analysis", model=MODELO_DISTIL, device=device)
print("Etiquetas del modelo:", clf_distil.model.config.id2label)
print()
pred_distil = mostrar_predicciones(clf_distil, nombre=MODELO_DISTIL)

# --- Los tres modelos sobre los mismos textos ---
def formatea(r):
    return f"{r['label']} ({r['score']:.2f})"

tabla_demo = pd.DataFrame({
    "Texto": [t[:55] + ("..." if len(t) > 55 else "") for t in TEXTOS_DEMO],
    "BETO (es)": [formatea(r) for r in pred_beto],
    "RoBERTuito (es)": [formatea(r) for r in pred_robertuito],
    "DistilBERT (en)": [formatea(r) for r in pred_distil],
})

print()
print("Los tres modelos sobre los mismos textos:")
print(tabla_demo.to_string(index=False))
```

```
Loading weights:   0%|          | 0/104 [00:00<?, ?it/s]
Etiquetas del modelo: {0: 'NEGATIVE', 1: 'POSITIVE'}

--- distilbert-base-uncased-finetuned-sst-2-english ---
  POSITIVE  0.967  <- El servicio fue excelente y el personal muy amable.
  NEGATIVE  0.766  <- Pesimo servicio, la habitacion estaba sucia y la comida fri
a.
  NEGATIVE  0.992  <- No recomiendo este hotel.
  NEGATIVE  0.587  <- El hotel esta bien, nada del otro mundo.
  NEGATIVE  0.984  <- El hotel impecable todo muy limpio las habitaciones amplias
y con todo
  POSITIVE  1.000  <- The service was excellent and the staff was very friendly.

Los tres modelos sobre los mismos textos:
                                                  Texto  BETO (es) RoBERTuito
(es) DistilBERT (en)
       El servicio fue excelente y el personal muy amable. POS (1.00)      POS (0.
98) POSITIVE (0.97)
Pesimo servicio, la habitacion estaba sucia y la comida... NEG (1.00)      NEG (0.
97) NEGATIVE (0.77)
                                No recomiendo este hotel. NEG (0.98)      NEG (0.
79) NEGATIVE (0.99)
                  El hotel esta bien, nada del otro mundo. POS (1.00)      POS (0.
87) NEGATIVE (0.59)
El hotel impecable todo muy limpio las habitaciones amp... POS (0.99)      POS (0.
94) NEGATIVE (0.98)
The service was excellent and the staff was very friend... POS (0.99)      POS (0.
94) POSITIVE (1.00)
```

In [30]:

```
# === Los tres modelos sobre el conjunto de prueba ===
# Cada modelo usa su propio esquema de etiquetas; se traduce al binario del corpu
# NEU -> "negativa" por coherencia con el criterio del Paso 1 (3 estrellas = nega
MAPEOS = {
    MODELO:            {"POS": "positiva", "NEU": "negativa", "NEG": "negativa"},
    MODELO_ROBERTUITO: {"POS": "positiva", "NEU": "negativa", "NEG": "negativa"},
    MODELO_DISTIL:     {"POSITIVE": "positiva", "NEGATIVE": "negativa"},
}

clasificadores = {
    MODELO: classifier,
    MODELO_ROBERTUITO: clf_robertuito,
    MODELO_DISTIL: clf_distil,
}

predicciones_por_modelo = {}
for nombre, clf in clasificadores.items():
    t0 = time.time()
    salida = clf(textos_test, batch_size=64 if usa_gpu else 16,
                 truncation=True, max_length=longitud_maxima(clf))
    mapeo = MAPEOS[nombre]
    predicciones_por_modelo[nombre] = pd.Series(
        [mapeo[p["label"]] for p in salida], index=y_test.index
    )
    print(f"{nombre:50s} {time.time()-t0:5.1f}s")

# --- Tabla final con los cinco modelos ---
comparacion_final = pd.DataFrame([
    resumen("LogReg base", y_test, y_pred),
    resumen("LogReg balanced", y_test, y_pred_bal),
    resumen("BETO (es)", y_test, predicciones_por_modelo[MODELO]),
    resumen("RoBERTuito (es)", y_test, predicciones_por_modelo[MODELO_ROBERTUITO]
    resumen("DistilBERT (en)", y_test, predicciones_por_modelo[MODELO_DISTIL]),
]).set_index("Modelo")

print()
print("Comparación final (clase 'negativa' = la minoritaria):")
print(comparacion_final.round(3).to_string())

print()
for nombre in (MODELO_ROBERTUITO, MODELO_DISTIL):
    print(f"--- {nombre} ---")
    print(classification_report(y_test, predicciones_por_modelo[nombre], digits=3
```

```
finiteautomata/beto-sentiment-analysis              28.5s
pysentimiento/robertuito-sentiment-analysis         17.7s
distilbert-base-uncased-finetuned-sst-2-english     19.0s

Comparación final (clase 'negativa' = la minoritaria):
                Accuracy  Precision (neg)  Recall (neg)  F1 (neg)  F1 macro
Modelo
LogReg base         0.952            0.873         0.673     0.760     0.867
LogReg balanced     0.937            0.661         0.910     0.766     0.865
BETO (es)           0.923            0.620         0.836     0.712     0.834
RoBERTuito (es)     0.946            0.724         0.850     0.782     0.875
DistilBERT (en)     0.276            0.135         0.994     0.238     0.274

--- pysentimiento/robertuito-sentiment-analysis ---
              precision    recall  f1-score   support

    negativa      0.724     0.850     0.782       786
    positiva      0.980     0.958     0.969      6127

    accuracy                          0.946      6913
   macro avg      0.852     0.904     0.875      6913
weighted avg      0.951     0.946     0.948      6913

--- distilbert-base-uncased-finetuned-sst-2-english ---
              precision    recall  f1-score   support

    negativa      0.135     0.994     0.238       786
    positiva      0.996     0.184     0.310      6127

    accuracy                          0.276      6913
   macro avg      0.565     0.589     0.274      6913
weighted avg      0.898     0.276     0.302      6913
```

In [31]:
```python
# === Matrices de confusión de los tres Transformers ===
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))

for ax, (titulo, nombre) in zip(axes, [
    ("BETO (es)", MODELO),
    ("RoBERTuito (es)", MODELO_ROBERTUITO),
    ("DistilBERT (en)", MODELO_DISTIL),
]):
    matriz = confusion_matrix(y_test, predicciones_por_modelo[nombre], labels=eti
    proporcion = matriz / matriz.sum(axis=1, keepdims=True)
    anotaciones = [
        [f"{matriz[i, j]:,}\n{proporcion[i, j]:.1%}" for j in range(matriz.shape[
        for i in range(matriz.shape[0])
    ]
    sns.heatmap(
        proporcion, annot=anotaciones, fmt="", cmap="Blues", vmin=0, vmax=1,
        cbar=False, square=True, linewidths=2, linecolor="white",
        xticklabels=[f"pred: {e}" for e in etiquetas],
        yticklabels=[f"real: {e}" for e in etiquetas],
        annot_kws={"fontsize": 10}, ax=ax,
    )
    ax.set_title(titulo, fontsize=12, pad=10)
    ax.tick_params(length=0)

fig.suptitle(
    "Transformers — el color es la proporción de cada fila (recall por clase)",
    fontsize=13,
)
fig.tight_layout()
plt.show()
```

